CS 423
Operating System Design: Concurrency: Locks
Mar 03

Ram Kesavan

Slide ack: Prof. Shivaram Venkataraman (Wisconsin)

ANNOUNCEMENTS

MP2 out; Due (3/23) 11:59 pm CT

MP1 grades: should be out later this week
    20% students: will be informed via email today

4Cr: next paper on “Mesa Monitors”, due 3/10

EXPECTATION: That students read OSTEP chapters

AGENDA / LEARNING OUTCOMES

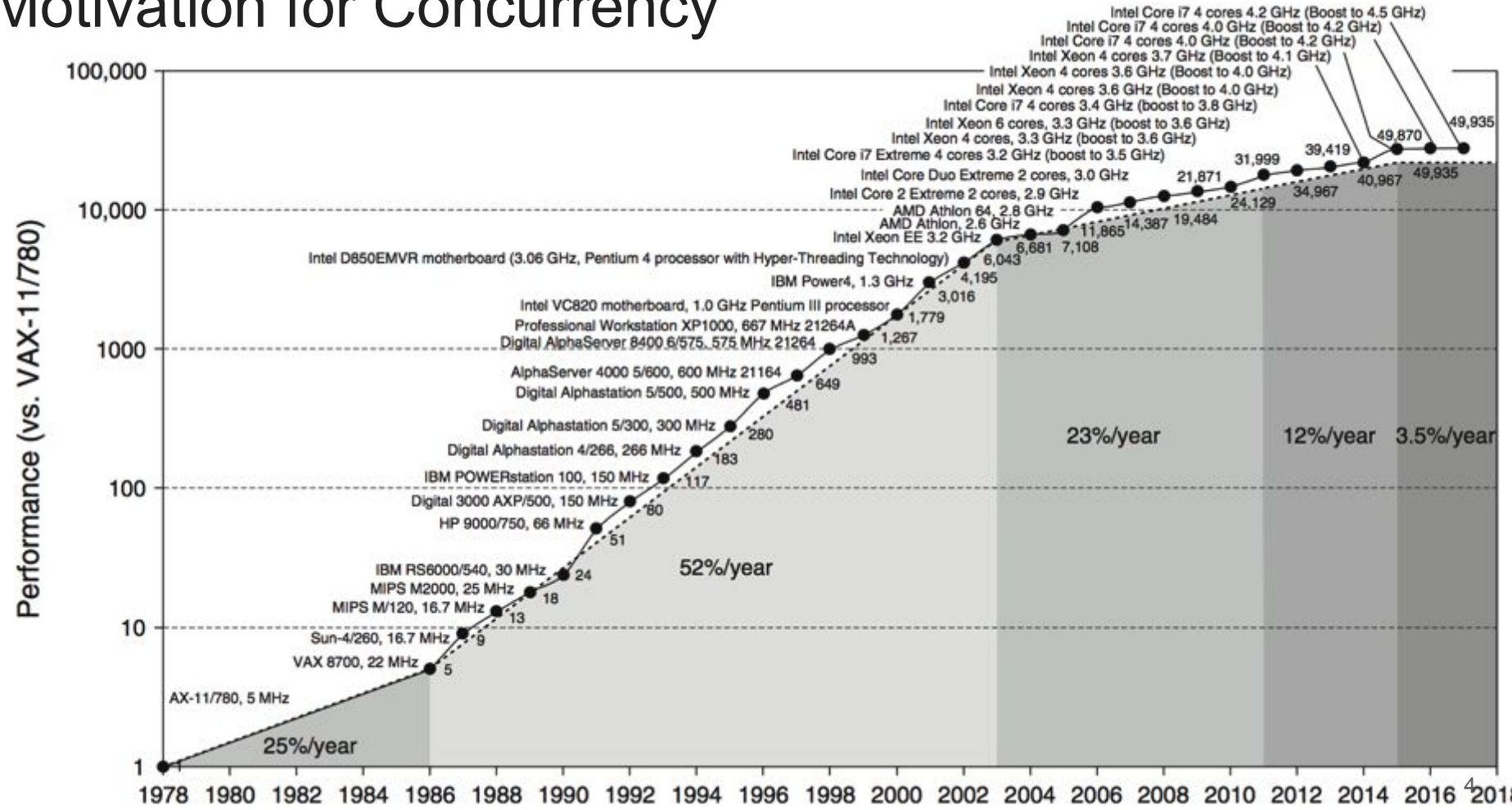
Concurrency:

- What is the motivation for concurrent execution
  - Challenges?
- Threading & Locks

Folks that have already taken 341 (or its equivalent)

- Some of this will be obvious; but we'll dig deeper

Motivation for Concurrency



MOTIVATION

CPU Trend: Speed has stabilized; increasing (maybe differentiated) #cores

Goal: Application utilize more cores

Option 1: Build app with many **processes**

- Example: Browser (process per tab)

Pros:

- Don't need new abstractions
- Good for security

Cons:

- High communication overhead
- Expensive context switching

Option 2: Build app with many **threads**

- Example: Word processors, video games

Pros:

- Shared address space
- Cheaper: fast context switch

Cons:

- Data corruption

Many modern apps are hybrid: use both options

THREADING

Threads (of same process) share address space

Each thread has its “own” stack

“own” -> but not really protected from other threads

Question: Walk through eg. of a bug where a thread corrupts another thread’s stack.

THREADING

New abstraction: thread

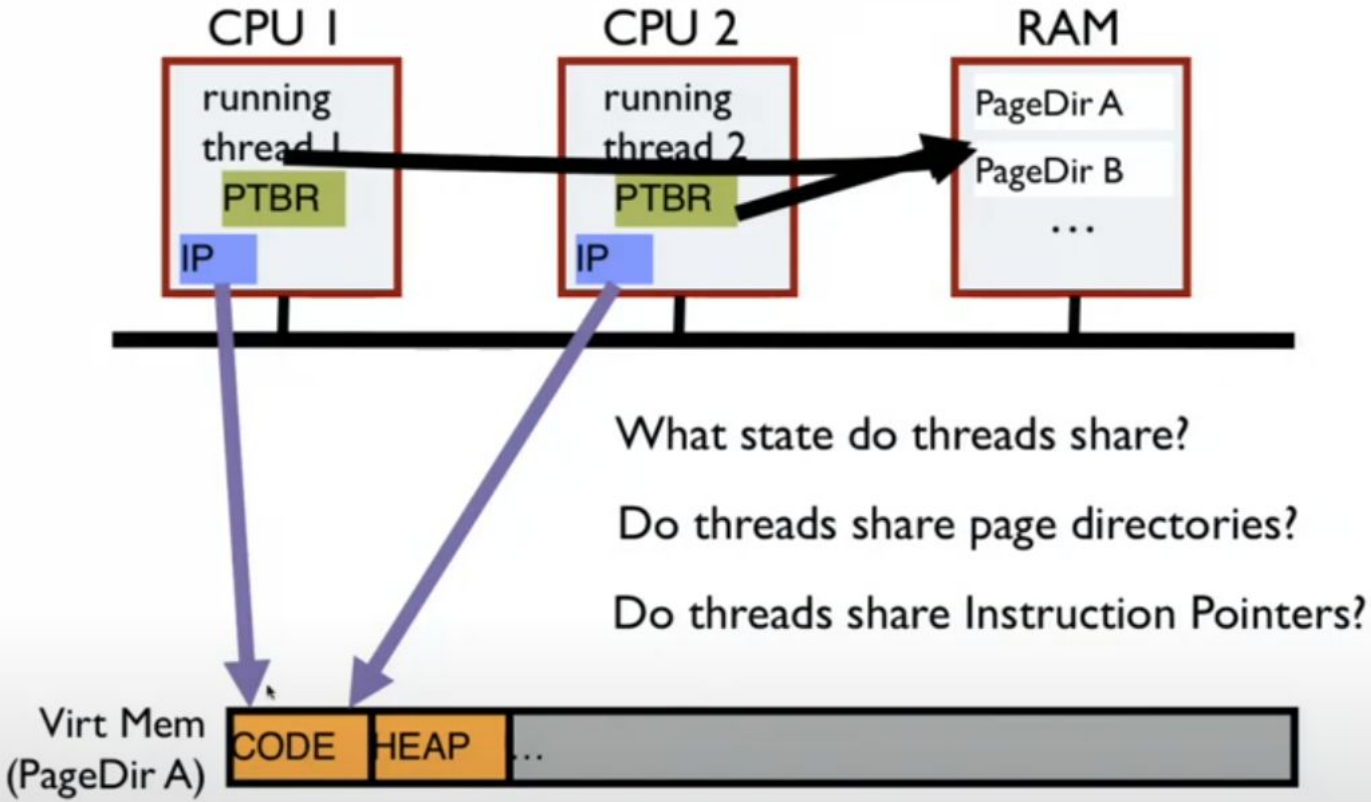
Threads are like “lightweight processes”, except:

- **Multiple threads (of same process) share the address space**
- **Each thread has its “own” stack**
  - “own” -> but not really protected from other threads

Inter-thread communication is cheaper (than inter-process communication)

- Don’t need to send the actual data because the heap is shared

In this course: POSIX thread library, aka **pthreads**

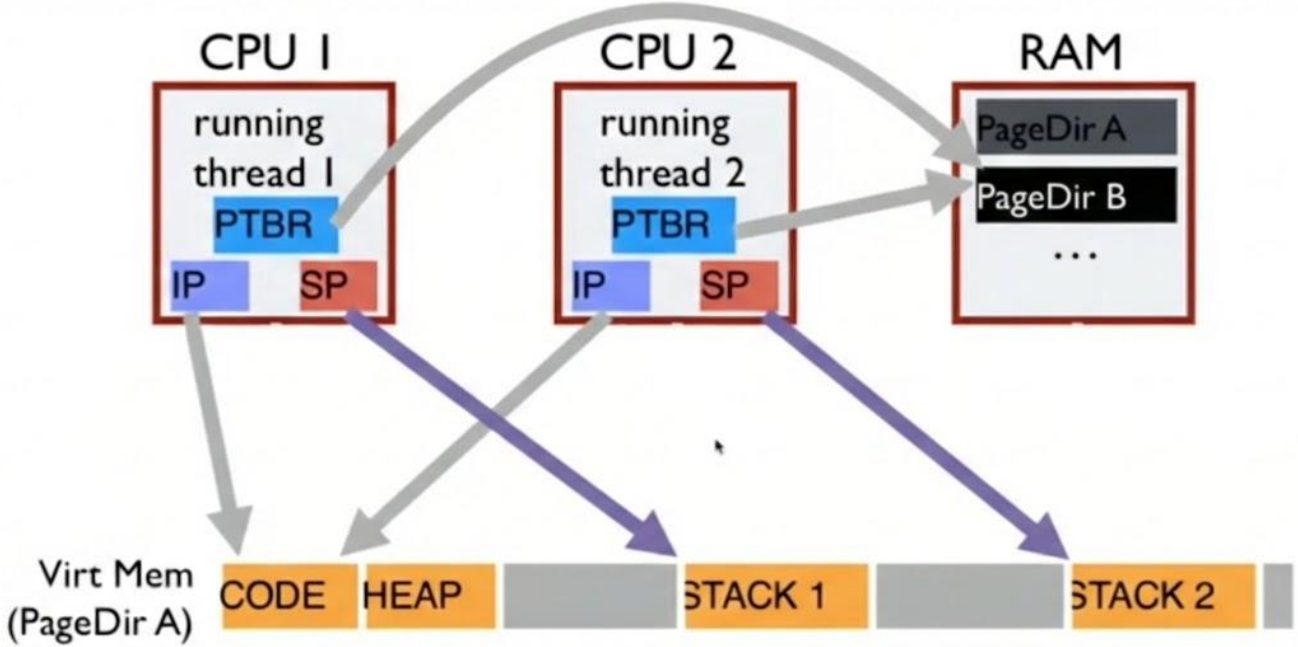


What state do threads share?

Do threads share page directories?

Do threads share Instruction Pointers?

Share code, but each thread may be executing different code at the same time
→ Different Instruction Pointers



Do threads share stack pointer?

Threads executing different functions need different stacks
(But, stacks are in same address space, so trusted to be cooperative)

THREADS WITHIN A PROCESS

All threads **share**:

- Process ID (PID)
- Page Table (PTBR), Address space: code (instructions), data (heap)
- Open file descriptors
- Current working directory
- User and group id

Each thread has its own **unique**:

- Thread ID (TID)
- Set of registers, including Program Counter and Stack Pointer
- Stack for local variables and return addresses (in same address space)

Threading: Approach 1

User-level threads: Many-to-one kernel thread mapping

- Implemented by user-level runtime libraries
- OS unaware of the existence of user-level threads
- OS believes each process contains only a single thread of control

Pros:

- No OS requirements; Completely portable
- Can tune policies to suit application
- Lower overhead switching; no system call to switch threads

Cons:

- Cannot leverage multiprocessors
- Entire process blocks when one thread blocks

Threading: Approach 2

Kernel-level threads: One-to-one thread mapping

- User-level runtime lib, but each user-level thread gets a kernel thread
- Each kernel thread schedulable independently
- OS performs thread operations (create, schedule, synchronize)

Pros:

- Multiple threads can run in parallel
- When a thread blocks, other threads are unaffected

Cons:

- Higher overhead for thread operations (compared to Approach 1)

THREAD SCHEDULE

```
volatile int balance = 0;
int loops = 1000000; // 1 million

void *worker(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
      balance++;
    }
    pthread_exit(NULL);
}
```

```
int main(int argc, char *argv[]) {
    pthread_t p1, p2;
    printf("Initial value : %d\n", balance);
    pthread_create(&p1, NULL, worker, NULL);
    pthread_create(&p2, NULL, worker, NULL);
    pthread_join(p1, NULL);
    pthread_join(p2, NULL);
    printf("Final value   : %d\n", balance);
    return 0;
}
```

EXPECTATION: Students will teach themselves pthreads

THREAD SCHEDULE #1

```
balance = balance + 1; balance at 0x9cd4
```

Registers are virtualized by OS;
Each thread thinks it has own

State:
0x9cd4: 100
%eax: ?
%rip = 0x195

process control blocks:

Thread 1	Thread 2
%eax: ? %rip: 0x195	%eax: ? %rip: 0x195

```
T1 ➡  0x195  mov 0x9cd4, %eax
      0x19a  add $0x1, %eax
      0x19d  mov %eax, 0x9cd4
```

THREAD SCHEDULE #1

State:
0x9cd4: 101
%eax: 101
%rip = 0x1a2

process control blocks:

Thread 1	Thread 2
%eax: ? %rip: 0x195	%eax: ? %rip: 0x195

```
0x195  mov 0x9cd4, %eax
0x19a  add $0x1, %eax
0x19d  mov %eax, 0x9cd4
```

T1

THREAD SCHEDULE #1

State:
0x9cd4: 101
%eax: ?
%rip = 0x195

process control blocks:

Thread 1	Thread 2
%eax: 101 %rip: 0x1a2	%eax: ? %rip: 0x195

T2

```
0x195  mov 0x9cd4, %eax
0x19a  add $0x1, %eax
0x19d  mov %eax, 0x9cd4
```

THREAD SCHEDULE #1

State:
0x9cd4: 102
%eax: 102
%rip = 0x1a2

process control blocks:

Thread 1	Thread 2
%eax: 101 %rip: 0x1a2	%eax: ? %rip: 0x195

```
0x195  mov 0x9cd4, %eax
0x19a  add $0x1, %eax
0x19d  mov %eax, 0x9cd4
```

T2 ➡

Desired Result!

THREAD SCHEDULE #2

State:
0x9cd4: 100
%eax: 101
%rip = 0x19d

process control blocks:

Thread 1	Thread 2
%eax: ?	%eax: ?
%rip: 0x195	%rip: 0x195

```
    0x195  mov 0x9cd4, %eax
    0x19a  add $0x1, %eax
T1  0x19d  mov %eax, 0x9cd4
```

Thread Context Switch before T1 executes 0x19d

THREAD SCHEDULE #2

State:
0x9cd4: 101
%eax: 101
%rip = 0x1a2

process control blocks:

Thread 1	Thread 2
%eax: 101 %rip: 0x19d	%eax: ? %rip: 0x195

```
0x195  mov 0x9cd4, %eax
0x19a  add $0x1, %eax
0x19d  mov %eax, 0x9cd4
```

T2

THREAD SCHEDULE #2

State:
0x9cd4: 101
%eax: 101
%rip = 0x1a2

process control blocks:

Thread 1	Thread 2
%eax: 101 %rip: 0x1a2	%eax: 101 %rip: 0x1a2

```
0x195  mov 0x9cd4, %eax
0x19a  add $0x1, %eax
0x19d  mov %eax, 0x9cd4
```

T1

WRONG Result! Final value of balance is 101

TIMELINE VIEW

Thread 1	**Thread 2**
mov 0x123, %eax	
add $0x1, %eax	
mov %eax, 0x123	
	mov 0x123, %eax
	add $0x2, %eax
	mov %eax, 0x123

TIMELINE VIEW

Thread 1	Thread 2	Balance	T1 %eax	T2 %eax
mov 0x123, %eax		0	0	
add $0x1, %eax		0	1	
mov %eax, 0x123		1	1	
	mov 0x123, %eax	1		1
	add $0x2, %eax			3
	mov %eax, 0x123	3		3

3 gets added to balance => correct behavior

TIMELINE VIEW

Thread 1	Thread 2	Balance	T1 %eax	T2 %eax
mov 0x123, %eax		0	0	
add $0x1, %eax		0	1	
	mov 0x123, %eax	0	1	
mov %eax, 0x123		1	1	0
	add $0x2, %eax			2
	mov %eax, 0x123	2		2

2 gets added to balance => incorrect behavior

2 MORE EXAMPLES

Thread 1	Thread 2
	mov 0x123, %eax
mov 0x123, %eax	
	add $0x2, %eax
add $0x1, %eax	
	mov %eax, 0x123
mov %eax, 0x123	

Thread 1	Thread 2
	mov 0x123, %eax
	add $0x2, %eax
mov 0x123, %eax	
add $0x1, %eax	
mov %eax, 0x123	
	mov %eax, 0x123

What happens in each case? Assume balance = 0 at the start.

NON-DETERMINISM

Concurrency leads to non-deterministic results

- Different results even with same inputs
- Race conditions

Whether a bug manifests depends on the CPU schedule.

How to program: imagine scheduler is behaving maliciously.

General principle: Assume the worst-case schedule!

WHAT DO WE WANT?

Want 3 instructions to execute as an un-interruptable group

That is, we want them to be **atomic**

```
mov 0x123, %eax
add $0x1, %eax
mov %eax, 0x123
```

More generally: need mutual exclusion for critical sections

If thread A is in the critical section, all other threads cannot

(okay if other threads do unrelated work)

Common Programming Models

Typical programming models for threaded programs:

- **Producer/consumer**
  Multiple producer threads create data (or work) that is handled by one of the multiple consumer threads
- **Pipeline**
  Task is divided into series of subtasks, each of which is handled in series by a different thread
- **Deferred work done in the background**
  One thread performs non-critical work in the background (when CPU idle)

Synchronization

Generic synchronization: for all models from previous slides

Goal: Learn how synchronization primitives are built (lib + OS)
With a little help from our friend…the hardware

Monitors
Locks
Semaphores
Condition Variables

Atomic Load
XCGH, Test&Set, etc.
Atomic Store
Disable Interrupts

LOCKS

Locks

Goal: Provide mutual exclusion (mutex)

Allocate and Initialize

- pthread_mutex_t mylock = PTHREAD_MUTEX_INITIALIZER;

Acquire

- Acquire exclusive access to lock
- Wait if lock is not available; some other thread has it
- Spin or block (relinquish CPU) while waiting
- pthread_mutex_lock(&mylock);

Release

- Release exclusive access to lock; let another thread at it
- pthread_mutex_unlock(&mylock);

LOCK IMPLEMENTATION GOALS

Goals:

- *Mutual exclusion*
  Only one thread in critical section at a time
- *Progress* (deadlock-free)
  If many simultaneous acquirers, must allow one to proceed
- *Bounded Waits* (starvation-free)
  Must eventually allow each waiting thread to enter

Fairness: Each acquirer waits for same amount of time

Performance: CPU is not wasted unnecessarily

IMPLEMENT LOCKS BY DISABLING INTERRUPTS

Turn off interrupts for critical sections
- Prevent timer interrupt, so no context switch possible
- Don't make any blocking system calls

Code between interrupts executes atomically

Can this work?

```
acquire(&lock);

// critical
//   section
//     code
//       here

release(&lock);
```

```
void acquire(lockT *l) {
    disableInterrupts();
}

void release(lockT *l) {
    enableInterrupts();
}
```

IMPLEMENT LOCKS w/ Load+Store

Use a single **shared** lock variable

```
// shared variable
boolean lock = false;
void acquire(Boolean *lock) {
    while (*lock); // spin…
    *lock = true;
}
void release(Boolean *lock) {
    *lock = false;
}
```

Does this work?

Implementing Synchronization

Disabling Interrupts

- Used by OS in very special circumstances
  - Eg. within an interrupt handler

Locking using loads/stores (Peterson's algorithm)

- A single instruction is indivisible (mostly)

```
mov 0x123, %eax      // word-aligned & sized memory…
mov %eax, 0x123      // …access is atomic
```

But locks today are built using support from hardware (and OS)

XCHG: ATOMIC EXCHANGE

How do we solve this ? Get help from the hardware!

```
// xchg(int *addr, int newval)
// return what was pointed to by addr
// at the same time, store newval into addr
int xchg(int *addr, int newval) {
        int old = *addr;
        *addr = newval;
        return old;
}
```

Variants of atomic operations: Test&Set, Compare&Swap

Test: ret old value, Set: set addr to new value
Compare: check value, if success then Swap: with new value

LOCK Implementation with XCHG

```
typedef struct __lock_t {
    int flag;
} lock_t;

void init(lock_t *lock) {
    lock->flag = 0;
}

void acquire(lock_t *lock) {
    ????;
    // spin-wait (do nothing)
}

void release(lock_t *lock) {
    lock->flag = ??;
}
```

int xchg(int *addr, int newval)

Using Atomic Instructions

```
typedef struct __lock_t {
    int flag;
} lock_t;

void init(lock_t *lock) {
    // 0: lock is available, 1: lock is held
    lock->flag = 0;
}

void lock(lock_t *lock) {
    while (TestAndSet(&lock->flag, 1) == 1)
        ; // spin-wait (do nothing)
}

void unlock(lock_t *lock) {
    lock->flag = 0;
}
```

Test – return old value

Set – set the passed in value

HW does them atomically!

Other Atomic HW Instructions

```
int CompareAndSwap(int *addr, int expected, int new) {
    int actual = *addr;
    if (actual == expected)
        *addr = new;
    return actual;
}
```

```
 void acquire(lock_t *lock) {
     while(CompareAndSwap(&lock->flag,  ,  ) ==  ) ;
     // spin-wait (do nothing)
 }
```

LOCK IMPLEMENTATION GOALS

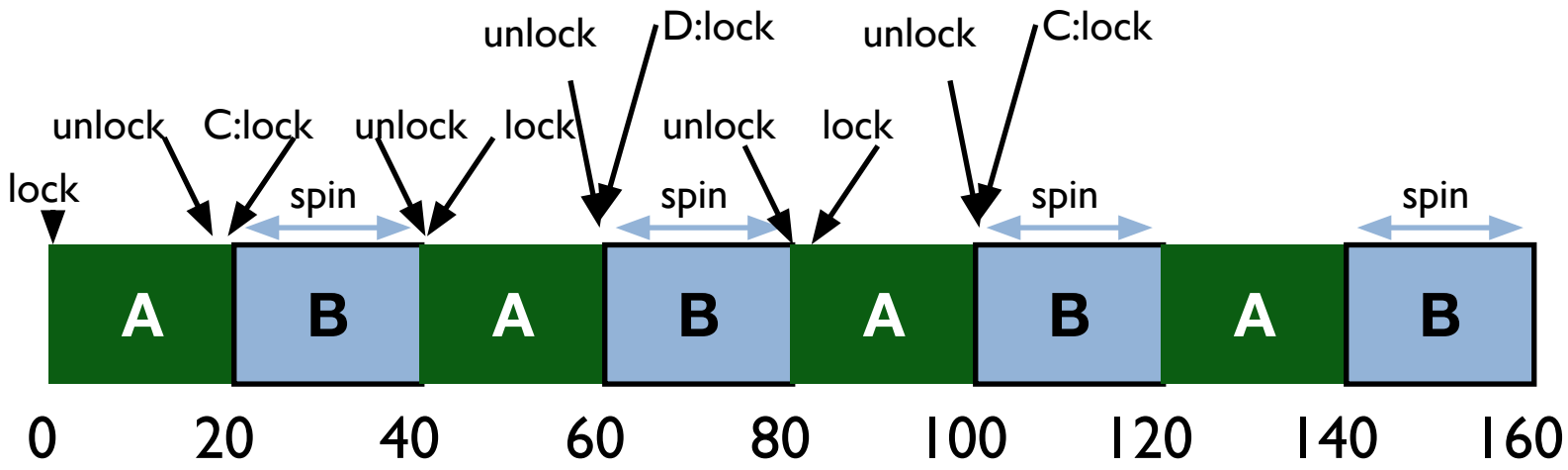
Correctness

- *Mutual exclusion (safety)*
  Only one thread in critical section at a time
- *Progress* (liveness)
  If several simultaneous requests, must allow one to proceed

Fairness: Does each thread have a fair shot at acquiring?
Does anybody starve?

Performance: Does CPU get wasted?

BASIC SPINLOCKS ARE UNFAIR



Scheduler is unaware of locks/unlocks!

B is unlucky - not guaranteed to acquire lock

FAIRNESS: TICKET LOCKS

Idea: reserve each thread's turn to use a lock.

Each thread spins until their turn.

Use new atomic primitive, fetch-and-add

```
int FetchAndAdd(int *ptr) {
    int old = *ptr;
    *ptr = old + 1;
    return old;
}
```

Acquire: Grab ticket; Spin while not thread's ticket != turn

Release: Advance to next turn

TICKET LOCK IMPLEMENTATION

```
typedef struct __lock_t {
    int ticket;
    int turn;
}

void lock_init(lock_t *lock) {
    lock->ticket = 0;
    lock->turn = 0;
}
```

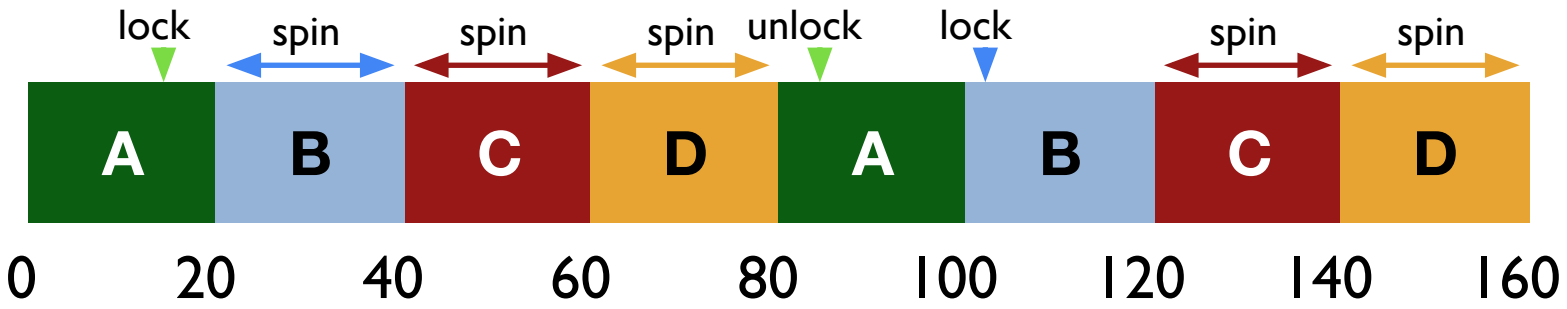
```
void acquire(lock_t *lock) {
    int myturn = FAA(&lock->ticket);
    while (lock->turn != myturn);
        // spin
}

void release(lock_t *lock) {
    lock->turn++;
}
```

SPINLOCK PERFORMANCE

Slow when…

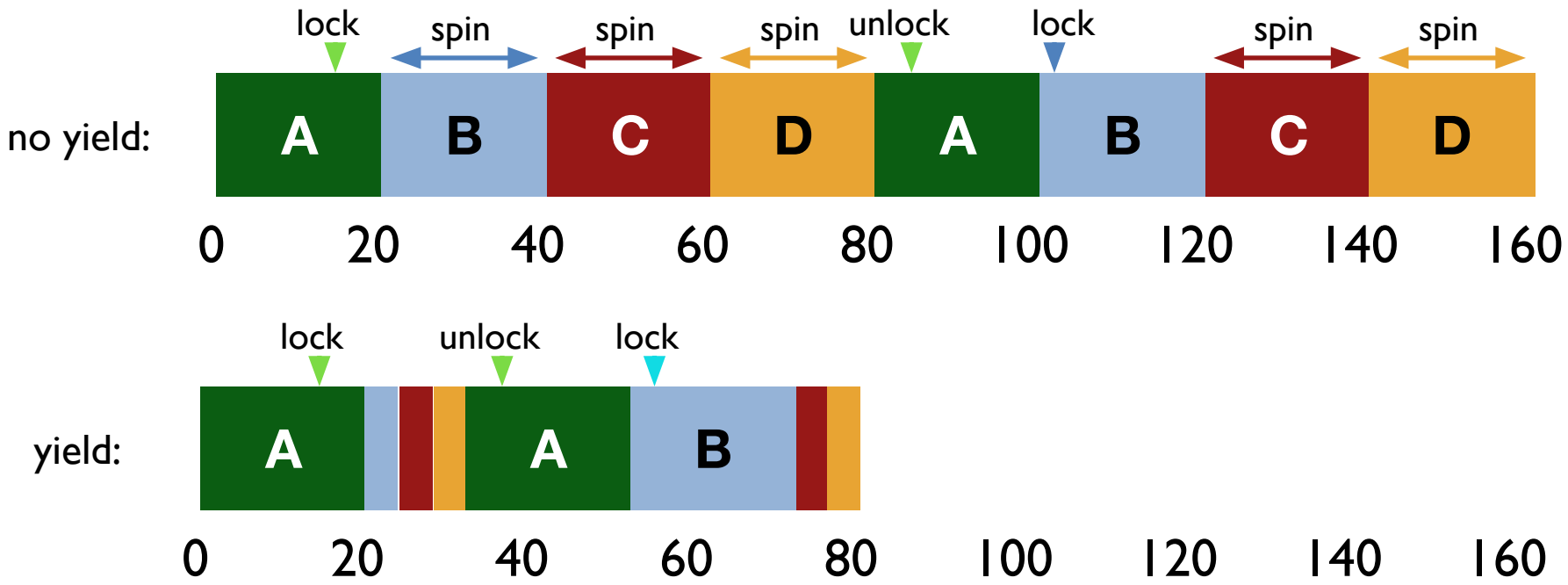
- one CPU
- locks held a long time
- disadvantage: spinning is wasteful

CPU SCHEDULER IS IGNORANT



CPU scheduler may run **B, C, D** instead of **A**
even though **B, C, D** are waiting for **A**

Yield Instead of Spin



YIELD VS SPIN

Waste of CPU cycles?

Without yield: O(threads * **time_slice**)

With yield: O(threads * **context_switch**)

While yield is better than spinning, with high contention, it can also be bad

Problem: the thread is still in READY state (and so can be scheduled on CPU)

Next improvement: Block and put thread on waiting queue

Lock Implementation: Block when Waiting

Remove waiting threads from scheduler runnable queue

Scheduler runs any thread that is **runnable**

Support in Solaris OS: park(), unpark()

Support in Linux: futex (fast userspace mutex)

HOLDING A LOCK: USER vs KERNEL

Question 1:
Can a thread running in user mode acquire a mutex, and then get context-switched or interrupted? If so, what happens?

Question 2:
Ditto for a thread running in kernel/privileged mode?

Question 3:
What if an interrupt tries to acquire a mutex?